

FINAL DIAMOND POLISH PROMPT

Hostile Audit, Bug Hunt, Balance Review, Optimization, and Gold-Master Packaging

The game is not considered final merely because it compiles or previously passed tests.

Reopen the current latest source as a hostile player, QA engineer, performance analyst, balance designer, release engineer, and external distributor.

Do not begin a new project.

Do not discard working features.

Do not assume previous “final,” “gold,” “diamond,” or “definitive” labels are accurate.

Your objective is to find and correct everything that would embarrass the project after public distribution.

1. SOURCE OF TRUTH

Use the latest complete source package as the source of truth.

Before editing:

1. Extract it into a new versioned work directory.
2. Verify its manifest.
3. Build it unchanged.
4. Run its current test suite.
5. Record the baseline.
6. Search for stale version strings and old release names.
7. Identify generated files that should not be edited manually.

Bump the version only after meaningful fixes are implemented and tested.

2. HOSTILE PLAYER WALKTHROUGH

Audit the complete player journey:

- first extraction
- first launch

- missing-file error
- title screen
- settings
- new session
- tutorial
- early progression
- midgame
- late game
- victory
- defeat
- return to title
- quit
- relaunch
- restored settings

For every state, ask:

- Is the objective obvious?
- Is the next action obvious?
- Can the player become soft-locked?
- Can capacity block a mandatory reward?
- Can a timer expire during an illegal state?
- Can text clip?
- Can images overlap?
- Can a click be lost?
- Can an optional file prevent startup?
- Can an action silently destroy value?
- Can the player accidentally abandon progress?
- Can the window appear off-screen?
- Can a missing monitor break fullscreen restoration?

3. BUG CLASSES TO SEARCH

Explicitly search for:

- stale cached state
- arrays overwritten through reused indexes
- virtual bonuses displayed as real ownership
- tie-breaking instability
- off-by-one capacity errors
- full inventory
- full bench
- full party
- full field
- chained upgrades

- reward overflow
- timeout overflow
- invalid targets
- dead actors continuing actions
- reflected damage killing a caster mid-resolution
- item loss
- duplicate item identity
- uninitialized optional assets
- mandatory files never loaded
- optional files treated as mandatory
- false UI state
- stale music titles
- stale hover state
- timer rounding
- timer clipping
- text-height mismatch
- unusual DPI
- fractional-resolution seams
- black bars
- redundant rendering
- simulation debt
- input starvation
- repeated fullscreen toggles
- repeated resize
- minimized execution
- focus loss
- settings corruption
- shutdown without saving

Add regression tests for every real defect found.

4. UI POLISH

Test at:

- 960×540 minimum
- 1600×900
- 1920×1080
- 2560×1440
- at least three awkward fractional resolutions
- borderless window
- borderless fullscreen
- maximized
- ultrawide letterbox
- high DPI

Validate:

- no black seams
- no exposed backbuffer
- no clipped headings
- no clipped long names
- no overlapping cards
- no invisible controls
- no hidden hitboxes
- no text beyond panel bounds
- no inconsistent padding
- no selection state hidden by portraits
- no important warning under another layer

Use measured text fitting.

Use shared transformed endpoints for adjacent regions.

Remove redundant panel and gradient passes.

Make timers, capacity warnings, upgrade opportunities, failure conditions, and overflow states unmistakable.

5. PERFORMANCE AUDIT

Profile conceptually and structurally:

- input path
- message pump
- fixed simulation
- render scheduling
- static UI
- animated UI
- combat
- asset loading
- audio
- resizing
- fullscreen transition

Find:

- per-frame allocation
- per-frame decoding
- repeated scaling
- repeated text measurement

- repeated resource creation
- redundant gradient fills
- identical scene rebuilds
- unbounded catch-up
- message-queue starvation
- forced synchronous repaint
- expensive inactive-window behavior

Maintain or improve:

- input immediacy
- stable simulation
- target presentation rate
- static-scene efficiency
- direct presentation
- high-DPI sharpness

Do not lower visual quality globally merely to hide a localized bottleneck.

6. BALANCE AUDIT

Do not rebalance everything because one encounter felt difficult.

Audit in layers:

Content curve

- power by cost/tier/level
- faction representation
- role representation
- rarity access
- damage profiles
- range advantage
- sustain
- defense
- crowd control
- economy

Stacking systems

- trait capstones
- faction capstones
- class capstones
- maximum patron/blessing levels
- items

- mutations
- global modifiers
- low-health comeback bonuses
- revival
- shielding
- healing
- omnivamp
- penetration

Prioritize excessive stacking over random unit nerfs.

Encounter curve

For every important encounter, record:

- enemy count
- clock
- power
- defenses
- abilities
- expected player power
- reward
- failure damage

Avoid raw-round formulas that create sudden walls.

Build deterministic simulations using legitimate evolving player states.

Sort them into low, average, and high-power bands.

Establish expected clear-rate bands.

Ordinary maintained builds should not lose primarily to a timer.

Strong builds should clear decisively.

Weak or greedy builds may fail meaningful checks.

Tutorial encounters should teach rather than punish.

Economy and progression

Verify:

- reward value matches risk
- upgrade costs

- reroll costs
- level timing
- interest
- streaks
- comeback potential
- no runaway first-place snowball
- no impossible recovery

Change only values supported by evidence.

7. AI AUDIT

Verify AI actually uses:

- normal shops
- normal currency
- paid rerolls
- paid progression
- items
- crafting
- positioning
- storage
- upgrades
- scouting
- pivots
- encounter rewards

Run long deterministic simulations.

Ensure opponents:

- differ across profiles
 - do not converge into one script
 - do not cheat
 - do not receive guaranteed rewards
 - do not destroy valuable upgraded units irrationally
 - maintain legal states
 - respond to competition
 - adapt to current systems
-

8. AUDIO AND ASSET FAILURE TESTS

Deliberately test:

- missing optional music
- missing optional voice
- missing optional ability cue
- missing mandatory background
- missing mandatory portrait
- corrupt audio
- corrupt image
- empty file

Optional content should fall back.

Mandatory content should produce an actionable error containing the first failed path.

Verify no file is loaded but unreachable.

Remove unreachable legacy assets.

Verify runtime package does not contain unused development debris.

9. SETTINGS AND LIFECYCLE

Verify:

- preferences save
 - preferences load
 - corrupt settings reset safely
 - outdated settings reset safely
 - window position restores
 - off-screen windows recover
 - fullscreen exits correctly
 - Escape priority is sensible
 - quitting active progress requires confirmation
 - clean shutdown releases resources
 - settings file is excluded from release ZIP
-

10. RELEASE ENGINEERING

Add or verify:

- embedded icon
- application manifest
- DPI awareness
- Windows file version
- product version
- description
- deterministic linking
- build guide
- player README
- controls
- troubleshooting
- release notes
- exact checksums
- verification scripts

Strip:

- old release notes
- old binaries
- old manifests
- object files
- import libraries
- test executables
- analyzer logs
- caches
- settings
- backups
- temporary archives

11. REQUIRED FINAL GATES

Run independently:

1. Strict release build with warnings as errors
2. Native behavior suite
3. Randomized stress suite
4. Balance simulation suite
5. Undefined-behavior sanitizer
6. Static analyzer
7. Asset decoder

8. Audio validator
9. UI contract checks
10. PE/resource inspection
11. Import inspection
12. Documentation consistency
13. ZIP integrity
14. SHA-256 manifest verification
15. Clean-room source rebuild
16. Clean-room retest
17. Byte-for-byte executable comparison when deterministic builds are supported

Do not allow an all-in-one wrapper quirk to conceal results.

Run problematic tools as isolated mandatory stages and verify fresh output timestamps.

12. FINAL RELEASE RESPONSE

Report:

- previous version
- final version
- every real bug found
- every meaningful polish improvement
- balance changes with evidence
- test totals
- simulation totals
- package file totals
- clean-room status
- deterministic build status
- SHA-256 values
- known limitations
- exact download links

Do not call the result final until these gates pass.